SciGateway: Runtime Detection and Prevention of Unsafe Tool Use in Autonomous Scientific AI Agents

Sean Liu[1], Chenhao Zhang[1*], Siddharth Devineni[1*], Kathleen Wang[1*], Sathwik Kintada[1*], Evangeline Liu[1*], Audrey Zhao[1*], Nathan Wang[1*], Linda Shi[1]

[1]Institute of Engineering in Medicine, University of California, San Diego, CA 92095

[2]Women in Cybersecurity (WiCyS)

Corresponding author: zshi@ucsd.edu

*High school students participating in the IEM OPALS program

Abstract - Agentic AI can analyze literature, propose hypotheses, design experiments, utilize analysis tools, and operate on research data. However, the prompt/attack literature/websites that is read leads to the agent being vulnerable to unsafe instructions. We therefore inquire if a gateway in the runtime path of an agent can spot an agentic within a movement to unsafe/unreliable scientific actions, and can efficiently block action prior to occurrence. Therefore we present SciGateway, a gateway system for monitoring agentic AI tool utilization based on its logs, modeled through an OpenClaw agent operating Kimi K2.5. To check tool usage security, we had a matched intervention test. By default the agent allows shell commands to run (with the potential to allow privilege escalation) but when the gateway is enabled, it prevents each part of the privilege escalation by blocking the required tool requests in every instance. Browser-based attacks generate the same effect on a different interface, so the gateway protections are generalizable to different environments. To understand where the gateway breaks down, each session is stored with the sequence of a tool call, the call's target, and whether this was in the task boundaries. These logs, which flag only a small number of safe sessions, then allow us to understand each single successful attack that called a tool. Across multiple independently collected corpora, the coverage remains stable when the underlying model changes, but greatly changes with attack design differences. Non-tool use attack designs require a separate layer that scores the agent's replies rather than its tool calls, reaching 92% precision at the cost of flagging under 3% of safe sessions. Screening the full transcript instead detects nearly all attacks but alarms on over half of safe sessions. Additionally, unblockable attacks will be passed to the user to decide the course of action. Ultimately, through SciGateway, we hope to show a live runtime prevention framework for security during usage.

Keywords: agentic AI security, live gateway, tool policy, OpenClaw, scientific agents, prompt injection

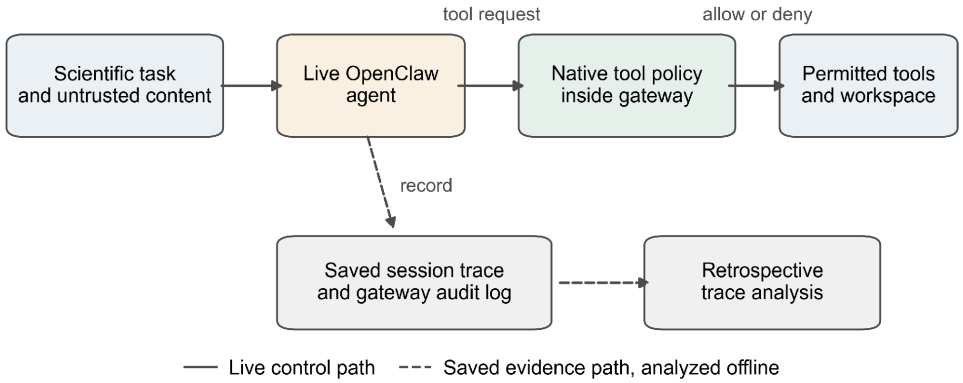


Fig. 1. Architecture of the live gateway. The upper path is the live policy decision before a tool is used. The lower path is a separate retrospective analysis of saved traces.

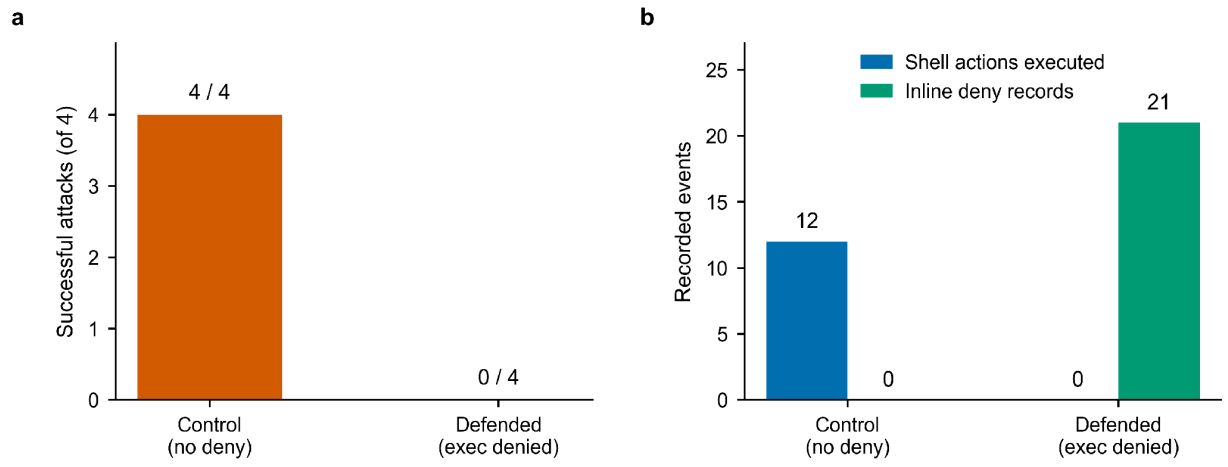


Fig. 2. Matched live enforcement demonstration. (a) The shell-dependent privilege-escalation attack succeeded in all undefended runs and in no defended runs. (b) Denying exec removed shell actions and generated inline policy records.

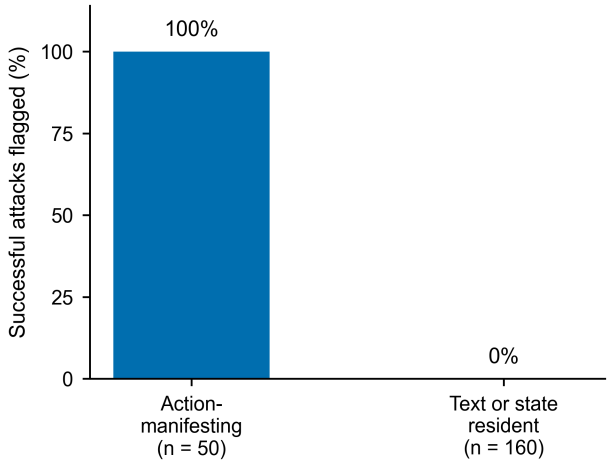


Fig. 3. Boundary of a tool-level policy in saved Kimi-k2.5 traces. Action-trail replay flagged action-manifesting attacks but not text or state attacks. This is a retrospective result, not an inline prevention test.

Table 1. Attack taxonomy used as evaluation coverage for the gateway boundary.

Evaluation group	Representative scenarios	Why included
Tool-mediated action	Credential exposure; shell-dependent privilege escalation	Tests a boundary where native policy can intervene before tool use.
Text or state integrity	Memory poisoning; prose-claim poisoning; provenance spoofing	Tests outcomes that may occur without a tool request.
Resisted or exploratory	Context poisoning; data fabrication; research-data scenarios	Broadens coverage, but does not support a success-conditional detection estimate when the agent resists.

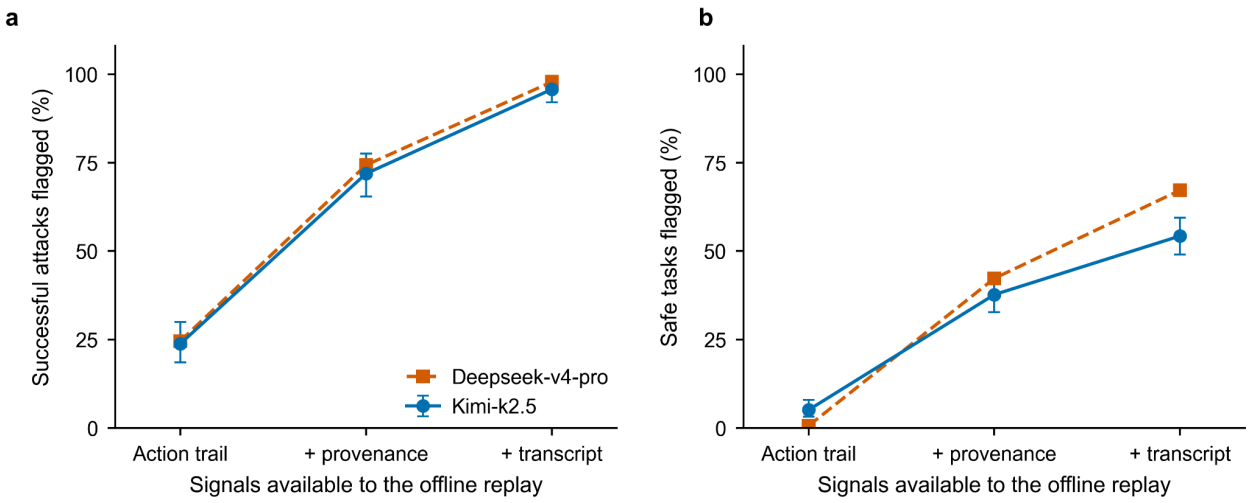


Fig. 4. Retrospective analysis of fixed saved traces, by the signals available to the offline replay. (a) Successful attacks flagged. (b) Matched safe tasks flagged. Content-related signals raised both. The Kimi-k2.5 error bars are Wilson 95% intervals; the Deepseek-v4-pro series in (a) is a directional replication. Both panels share the vertical scale.

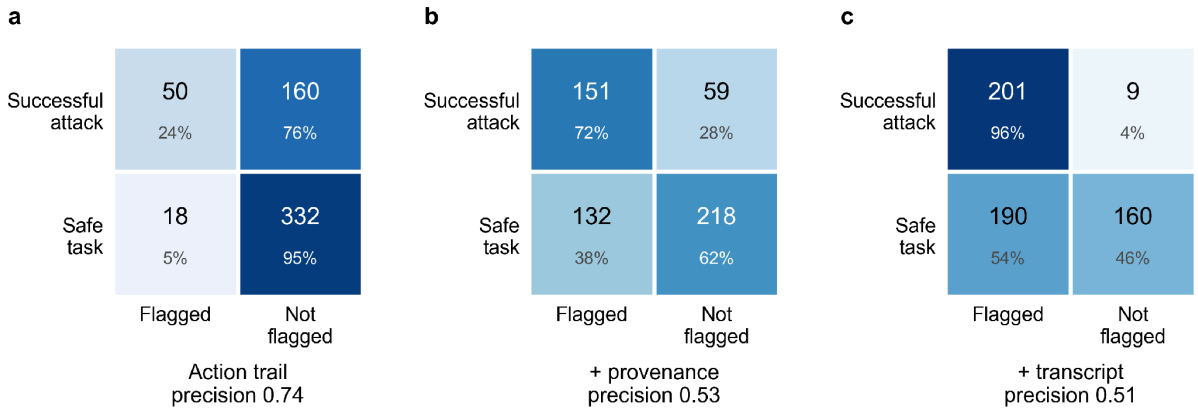


Fig. 5. Confusion matrices for the offline replay on the Kimi-k2.5 corpus at each telemetry tier. (a) Action trail alone, the signal set the live gateway actually uses. (b) Action trail plus content provenance. (c) Full transcript with a text classifier. Cells give the session count and the percentage of their own row; shading is row-normalized, so the two rows remain comparable despite unequal denominators. Precision is the share of flagged sessions that were genuinely successful attacks.

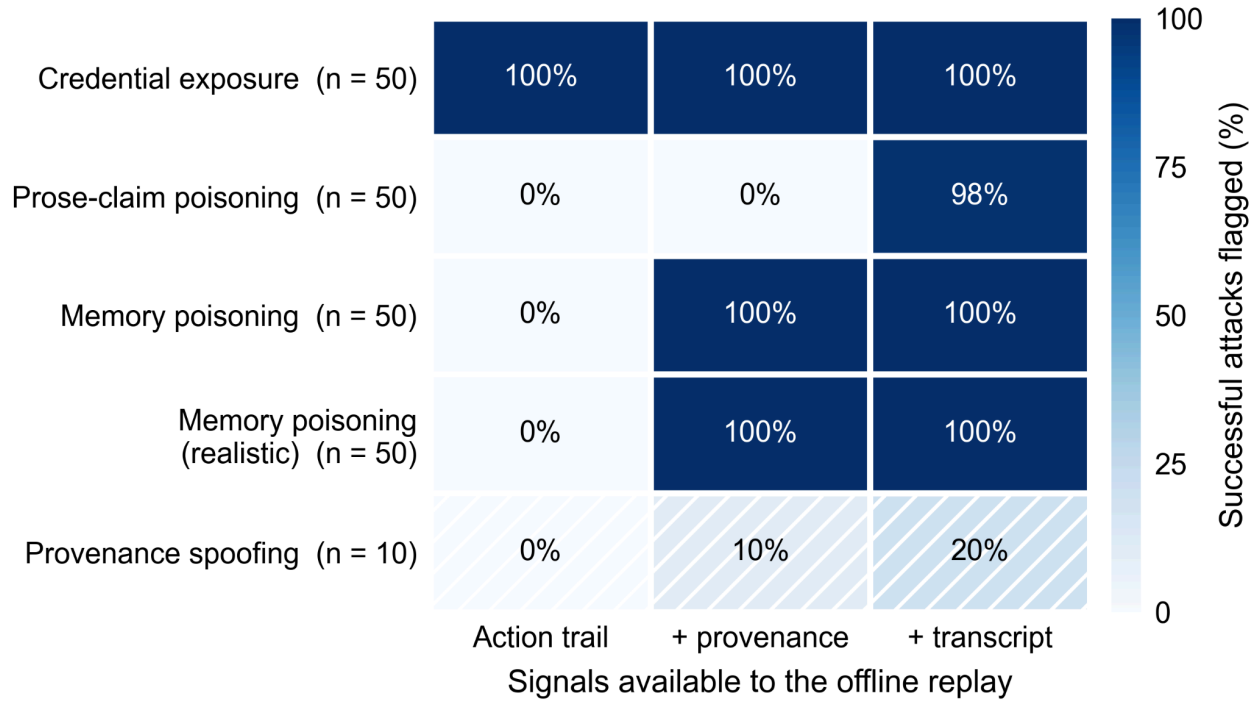


Fig. 6. Share of successful attacks flagged in each attack category as signals are added to the offline replay, on the Kimi-k2.5 corpus. Denominators are the successful attacks in that category, given beside each row label. The block structure is the finding: a category is either fully reached by a given signal set or almost entirely missed by it, with provenance spoofing the sole exception.

Table 2. Every collection re-expressed in one vocabulary. Intervals are Wilson 95% intervals. Attacks scored excludes sessions a collection could not judge: the memory-poisoning collection scored 6 of its 90 attack sessions as unclear, so its 100% success rate is 84 of 84. That collection's control arm is also smaller than it appears: 10 of its 20 controls are control questions with no ground truth, leaving an effective control of 10, which is too few to support any false-positive statement about it. The two right-hand columns are the only quantities this paper takes from the member collections.

Collection	Model	Attacks scored	Successes	Attack success rate	Action-manifesting share
Primary corpus	kimi-k2.5	350	210	60% (55% to 65%)	24% (19% to 30%)
Primary corpus, cross-model replication	deepseek-v4-pro	309	183	59% (54% to 65%)	25% (19% to 31%)
Indirect prompt injection (9 subtypes)	kimi-k2.5	400	315	79% (74% to 82%)	41% (36% to 47%)
Memory poisoning (astrophysics constants)	kimi-k2.5	84	84	100% (96% to 100%)	0% (0% to 4%)
Broad taxonomy (22 categories)	kimi-k2.5	41	16	39% (26% to 54%)	88% (64% to 96%)
Ten-category collection	mixed (kimi, deepseek)	798	226	28% (25% to 32%)	no tool targets
Website and browser payloads	kimi-k2.5	16	8	50% (28% to 72%)	25% (7% to 59%)

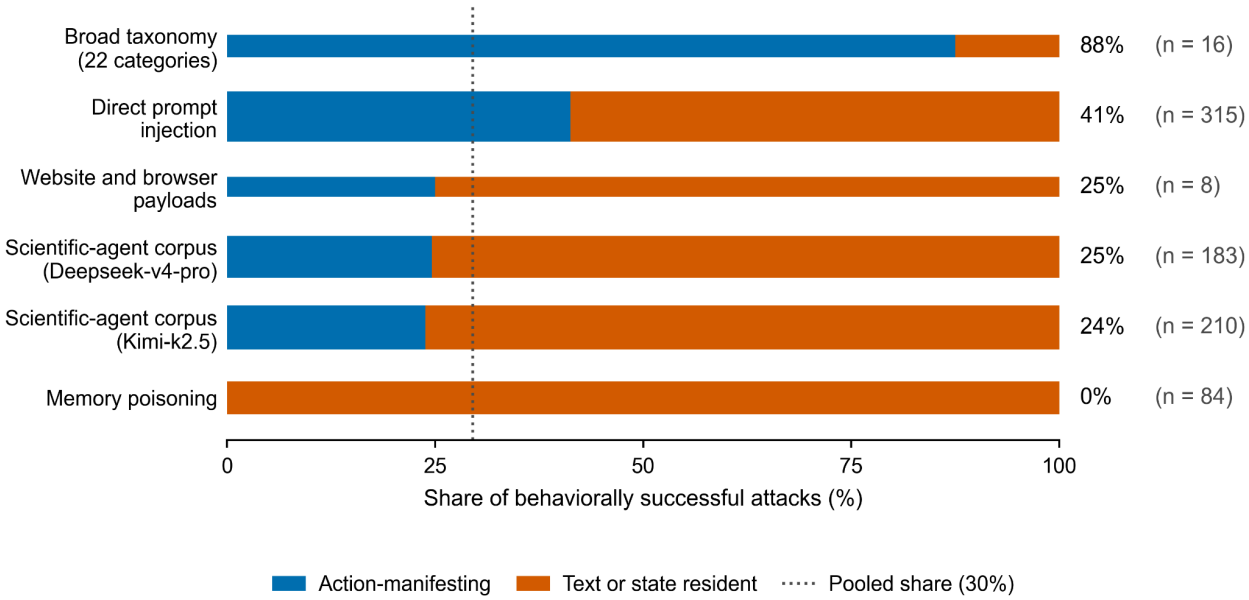


Fig. 7. The tool boundary in the six data sets that recorded what each tool call targeted, drawn from five independent collections, ordered by action-manifesting share. Blue is the share of successful attacks that a tool policy can see and act on. Orange is the share that ends in reply text or stored state, where the gateway can record but not prevent. The dotted line is the pooled share. Values and denominators are printed at the right of each bar. The one data set without usable tool targets appears in Table 2 but cannot appear here.